

Unit Testing (JUnit 5) - WhiteBoxSample

1. Testing Framework

Item	Value
Framework	JUnit 5.10.1
Language	Java 17
Build Tool	Maven
Coverage Tool	JaCoCo

2. Test File Structure

```
java-testing/  
  pom.xml  
  src/  
    main/java/vv/sample/  
      WhiteBoxSample.java  
    test/java/vv/sample/  
      WhiteBoxSampleTest.java
```

3. Test Cases Summary (35+ Tests)

3.1 Account Class Tests (11 Tests)

Test ID	Category	Description	Status
TC-ACC-01	Normal	Create account with valid data	☐
TC-ACC-02	Exception	Null ID throws exception	☐
TC-ACC-03	Exception	Blank ID throws exception	☐
TC-ACC-04	Exception	Negative balance throws exception	☐
TC-ACC-05	Boundary	Zero balance succeeds	☐
TC-ACC-06	Normal	Deposit increases balance	☐
TC-ACC-07	Exception	Non-positive deposit throws	☐
TC-ACC-08	Exception	Deposit to frozen account throws	☐
TC-ACC-09	Normal	Withdraw decreases balance	☐
TC-ACC-10	Exception	Insufficient funds throws	☐
TC-ACC-11	Normal	Freeze/unfreeze works	☐

3.2 TransactionProcessor Tests (9 Tests)

Test ID	Category	Description	Status
TC-TXN-01	Invalid	Null source returns error	☐
TC-TXN-02	Invalid	Null destination returns error	☐
TC-TXN-03	Invalid	Frozen source returns error	☐
TC-TXN-04	Invalid	Frozen destination returns error	☐
TC-TXN-05	Invalid	Zero amount returns error	☐

Test ID	Category	Description	Status
TC-TXN-06	Invalid	Negative amount returns error	☐
TC-TXN-07	Normal	Successful VIP transfer	☐
TC-TXN-08	Edge	Insufficient funds after fee	☐
TC-TXN-09	Normal	Log updated after transfer	☐

3.3 calculateFee Tests (8 Tests)

Test ID	Category	Description	Input	Expected	Status
TC-FEE-01	Normal	VIP fee > 1000	VIP, 2000	10.0	☐
TC-FEE-02	Normal	VIP fee <= 1000	VIP, 500	5.0	☐
TC-FEE-03	Boundary	VIP at 1000	VIP, 1000	10.0	☐
TC-FEE-04	Normal	PREMIUM fee	PREMIUM, 1000	15.0	☐
TC-FEE-05	Normal	REGULAR fee >= 50	REGULAR, 100	2.0	☐
TC-FEE-06	Edge	REGULAR fee < 50	REGULAR, 30	1.0	☐
TC-FEE-07	Boundary	REGULAR at 50	REGULAR, 50	1.0	☐
TC-FEE-08	Edge	Zero/negative amount	VIP, 0	0.0	☐

3.4 SecurityUtils.isPasswordStrong Tests (7 Tests)

Test ID	Category	Description	Input	Expected	Status
TC-PWD-01	Invalid	Null password	null	false	☐
TC-PWD-02	Boundary	Less than 8 chars	"Abc\$12"	false	☐
TC-PWD-03	Boundary	Exactly 8 chars strong	"Abcd\$123"	true	☐
TC-PWD-04	Edge	Only 2 types	"abcdABCD"	false	☐
TC-PWD-05	Normal	3 types	"Abcdefg1"	true	☐
TC-PWD-06	Normal	All 4 types	"Abc\$1234"	true	☐
TC-PWD-07	Edge	Only digits	"12345678"	false	☐

3.5 SecurityUtils.sanitize Tests (5 Tests)

Test ID	Category	Description	Input	Expected	Status
TC-SAN-01	Invalid	Null input	null	null	☐

| TC-SAN-02 | Normal | Escape < | "

" | "<script>" | ☐ | | TC-SAN-03 | Normal | Escape > | "test>val" | "test>val" | ☐ | | TC-SAN-04 | Normal | Trim whitespace | " hello " | "hello" | ☐ | | TC-SAN-05 | Normal | Plain text | "Hello" | "Hello" | ☐ |

3.6 DiscountCalculator Tests (10 Tests)

Test ID	Category	Description	Status
TC-DSC-01	Exception	Negative price throws	☐
TC-DSC-02	Normal	VIP base 10%	☐

Test ID	Category	Description	Status
TC-DSC-03	Normal	PREMIUM base 5%	☐
TC-DSC-04	Normal	REGULAR base 0%	☐
TC-DSC-05	Normal	Loyalty > 20 = 3%	☐
TC-DSC-06	Normal	Loyalty 11-20 = 1%	☐
TC-DSC-07	Normal	No loyalty <= 10	☐
TC-DSC-08	Edge	Seasonal VIP bonus	☐
TC-DSC-09	Normal	Mid-price bonus	☐
TC-DSC-10	Boundary	Discount capped at 25%	☐

4. Test Distribution by Category

Category	Count	Percentage
Normal	18	51%
Exception/Invalid	10	29%
Edge	5	14%
Boundary	5	14%
Total	35+	100%

5. Running Tests

Using Maven:

Navigate to java-testing directory

```
cd java-testing
```

Run all tests

```
mvn test
```

Run tests with coverage report

```
mvn test jacoco:report
```

View coverage report

```
open target/site/jacoco/index.html
```

Using IDE:

1. Open java-testing folder in IntelliJ IDEA or Eclipse
2. Right-click on WhiteBoxSampleTest.java
3. Select "Run with Coverage"

6. Sample Test Code

```
@Nested
@DisplayName("Account Class Tests")
```

```

class AccountTests {

    @Test
    @DisplayName("TC-ACC-01: Create account with valid data")
    void testCreateValidAccount() {
        Account acc = new Account("A001", 1000.0, CustomerType.VIP);
        assertEquals("A001", acc.getId());
        assertEquals(1000.0, acc.getBalance());
        assertEquals(CustomerType.VIP, acc.getType());
        assertFalse(acc.isFrozen());
    }

    @Test
    @DisplayName("TC-ACC-02: Create account with null ID throws exception")
    void testCreateAccountNullId() {
        assertThrows(IllegalArgumentException.class, () -> {
            new Account(null, 1000.0, CustomerType.REGULAR);
        });
    }

    @Test
    @DisplayName("TC-ACC-10: Withdraw more than balance throws exception")
    void testWithdrawInsufficientFunds() {
        Account acc = new Account("A001", 100.0, CustomerType.REGULAR);
        assertThrows(IllegalArgumentException.class, () -> acc.withdraw(150.0));
    }
}

```

7. Expected Coverage Results

Class	Line Coverage	Branch Coverage
Account	100%	100%
TransactionProcessor	100%	100%
SecurityUtils	100%	100%
DiscountCalculator	95%+	100%
Overall	97%+	100%

8. Test Annotations Used

Annotation	Purpose
@Test	Mark method as test
@DisplayName	Human-readable test name
@Nested	Group related tests
@BeforeEach	Setup before each test
@ParameterizedTest	Data-driven tests

9. Assertions Used

Assertion	Purpose
<code>assertEquals()</code>	Verify expected equals actual
<code>assertTrue()</code>	Verify condition is true
<code>assertFalse()</code>	Verify condition is false
<code>assertNull()</code>	Verify value is null
<code>assertThrows()</code>	Verify exception is thrown